# Boltzmannator

*A Visual Explorer for One-Dimensional Normalising Flows*

Claude Code[1] and Christoph Dellago[2]

[1]Anthropic
[2]Faculty of Physics, University of Vienna

## Contents



## 1 Introduction

**Boltzmannator** is an interactive, browser-based application for exploring normalising flows in one dimension. Given a user-chosen *latent* distribution $p_z(z)$ and a parametric transformation

$x = f_\theta(z)$, the app visualises how the probability density is reshaped under the map, computes the push-forward density $p_x(x)$, and trains the parameters $\theta$ against either a target Boltzmann distribution (energy-based mode) or a set of example data points (maximum-likelihood mode). The aim is to make the interplay between the transformation, the Jacobian, and the resulting density immediately tangible.

Normalising flows were introduced as a tool for flexible probability-density modelling and variational inference [1]; see [2] for a comprehensive review. In statistical physics the same idea underlies *Boltzmann Generators*, which learn a flow that maps a simple latent density onto a target Boltzmann distribution to sample equilibrium states of many-body systems [3] (see [4] for a short review). Boltzmann Generators have since been extended to free-energy estimation via learned mappings [5], to rare-event and transition-path sampling [6], to the efficient mapping of phase diagrams and equilibrium sampling of large-scale materials with conditional flows [7, 8], and to learning mappings between equilibrium states of liquid systems [9]. Boltzmannator illustrates these ideas in the simplest possible one-dimensional setting.

The program is written in Python using the NiceGUI framework, with all numerics in NumPy and the figures rendered by Matplotlib. It runs as a small local web server and is used through a web browser at `http://localhost:8080`. Because it is served over the network, several users on the same local network can connect simultaneously (e.g. students in a classroom opening `http://`*`host-ip`*`:8080`); each browser gets its own fully independent session, so one user's parameters and training never affect another's. Installation and start-up instructions are given in the accompanying `README`.

# 2 Mathematical Background

## 2.1 The Change-of-Variables Formula

Let $z \in \mathbb{R}$ be a random variable with density $p_z(z)$, and let $x = f_\theta(z)$ be a differentiable, invertible (monotone) function parametrised by $\theta$. The *push-forward* density of $x$ is obtained from the change-of-variables formula:

$$p_x(x) \;=\; \frac{p_z\big(f_\theta^{-1}(x)\big)}{|J_f(z)|} \;=\; \frac{p_z(z)}{|J_f(z)|}, \tag{1}$$

where $z = f_\theta^{-1}(x)$ is the pre-image and

$$J_f(z) \;=\; \frac{df_\theta}{dz} \tag{2}$$

is the (scalar) Jacobian. Equation (1) is the cornerstone of normalising flows: by choosing $f_\theta$ appropriately, any latent density $p_z$ can be mapped to an arbitrarily complex target density $p_x$.

The app displays the four quantities in (1) simultaneously: $p_z(z)$ (top strip), the curve $x = f_\theta(z)$ (main panel), $|J_f(z)|^{-1}$ (bottom strip), and $p_x(x)$ (right panel). The bottom strip shows the *inverse* Jacobian because $p_x(x) = p_z(z) \times |J_f(z)|^{-1}$: the value directly reads off the local density-magnification factor ($> 1$ compresses, $< 1$ dilutes).

## 2.2 Latent Distributions

The following latent densities are available, all parametrised by a mean $\mu$ and a scale $\sigma$:

| Name | Density $p_z(z)$ | Standard deviation |
|---|---|---|
| Gaussian | $\frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(z-\mu)^2}{2\sigma^2}\right)$ | $\sigma$ |
| Uniform | $\frac{1}{2\sigma\sqrt{3}}$ for $\lvert z-\mu\rvert \le \sigma\sqrt{3}$, else 0 | $\sigma$ |
| Laplace | $\frac{1}{\sigma\sqrt{2}} \exp\left(-\frac{\sqrt{2}\,\lvert z-\mu\rvert}{\sigma}\right)$ | $\sigma$ |
| Bimodal | $\frac{1}{2}\mathcal{N}(z;-\mu,\sigma) + \frac{1}{2}\mathcal{N}(z;+\mu,\sigma)$ | $\sqrt{\mu^2+\sigma^2}$ |

For the **Bimodal** distribution the slider $\mu$ sets the *peak positions* at $\pm\mu$ (not the overall mean, which is always zero), while $\sigma$ sets the width of each individual peak. The slider is therefore restricted to $\mu \ge 0$; the default values are $\mu = 1$, $\sigma = 0.5$.

### 2.3 The Target Boltzmann Distribution

The target density used for energy-based training is a one-dimensional Boltzmann distribution,

$$p^*(x) \;=\; \frac{1}{Z} \exp\left(-\frac{U(x)}{k_B T}\right), \qquad Z = \int_{-\infty}^{\infty} \exp\left(-\frac{U(x)}{k_B T}\right) dx, \tag{3}$$

with the polynomial potential energy

$$U(x) \;=\; u_1 x + u_2 x^2 + u_3 x^3 + u_4 x^4. \tag{4}$$

The coefficients $u_1, \ldots, u_4$ and the temperature $T$ are controlled via sliders. A positive $u_4$ ensures that $U \to +\infty$ for $|x| \to \infty$ and that the partition function $Z$ is finite.

#### Exact transformation

When the target distribution is set up, the theoretically perfect transformation $x^*(z)$ can be computed as the composition of CDFs:

$$x^*(z) \;=\; F_{p^*}^{-1}\big(F_{p_z}(z)\big), \tag{5}$$

where $F_{p_z}$ and $F_{p^*}$ are the cumulative distribution functions of the latent and target densities, respectively. This *exact* curve is shown as a dashed reference line in the transformation panel. Toggling *Show exact transformation* does not change the displayed $z$- or $x$-range (the curve is simply drawn, clipped to the current axes).

## 3 Parametric Transformations

### 3.1 Polynomial Transformation

The polynomial family is

$$f_\theta(z) \;=\; \theta_0 + \theta_1 z + \theta_2 z^2 + \theta_3 z^3, \tag{6}$$

with Jacobian

$$J_f(z) \;=\; \theta_1 + 2\theta_2 z + 3\theta_3 z^2. \tag{7}$$

A cubic can represent shifts, scalings, skewness, and mild non-linearity, but it is not guaranteed to be monotone: if $J_f$ changes sign the transformation is not invertible and the change-of-variables formula (1) does not apply. The app highlights regions of non-monotonicity in orange and displays a warning in the main panel.

### 3.2 Rational-Quadratic Spline (RQS) Transformation

The RQS family [10] defines a piecewise rational-quadratic bijection on a finite support $[-B, B]$ with $K$ bins, and extends linearly (identity) outside the support.

**Parameters.** Given $K$ bins, the raw parameters are:

- Boundary $B > 0$ (one slider).
- Unconstrained width logits $\tilde{w}_1, \ldots, \tilde{w}_K$: actual bin widths $\Delta x_k = 2B \cdot \mathrm{softmax}(\tilde{w})_k$, so $\sum_k \Delta x_k = 2B$.
- Unconstrained height logits $\tilde{h}_1, \ldots, \tilde{h}_K$: actual heights $\Delta y_k = 2B \cdot \mathrm{softmax}(\tilde{h})_k$.
- Unconstrained derivative logits $\tilde{d}_0, \ldots, \tilde{d}_K$: actual knot derivatives $d_k = e^{\tilde{d}_k} > 0$. Setting $\tilde{d}_k = 0$ gives $d_k = 1$.

Knot positions are $x_0 = y_0 = -B$ and $x_{k+1} = x_k + \Delta x_k$, $y_{k+1} = y_k + \Delta y_k$. The total number of parameters is $3K + 2$ (including $B$).

**Forward pass.** For $z$ in bin $k$ (i.e. $x_k \le z < x_{k+1}$), define $\xi = (z - x_k)/\Delta x_k \in [0, 1]$ and $s_k = \Delta y_k / \Delta x_k$. Then

$$f_\theta(z) \;=\; y_k \;+\; \Delta y_k \, \frac{s_k \, \xi^2 + d_k \, \xi(1-\xi)}{s_k + (d_{k+1} + d_k - 2s_k)\, \xi(1-\xi)}, \tag{8}$$

with Jacobian

$$J_f(z) \;=\; \frac{s_k^2 \big[ d_{k+1} \, \xi^2 + 2 s_k \, \xi(1-\xi) + d_k (1-\xi)^2 \big]}{\big[ s_k + (d_{k+1} + d_k - 2s_k)\, \xi(1-\xi) \big]^2}. \tag{9}$$

Since all $\Delta x_k, \Delta y_k, d_k > 0$ the transformation is always strictly monotone ($J_f > 0$ everywhere in the support).

**Inverse pass.** Given $x_{\text{out}}$ in bin $k$ (located by $y_k \le x_{\text{out}} < y_{k+1}$), the normalised position $\xi$ is the root of the quadratic $a\xi^2 + b\xi + c = 0$ with

$$a = \Delta y_k (s_k - d_k) + (x_{\text{out}} - y_k)(d_{k+1} + d_k - 2s_k), \tag{10}$$

$$b = \Delta y_k \, d_k - (x_{\text{out}} - y_k)(d_{k+1} + d_k - 2s_k), \tag{11}$$

$$c = -s_k (x_{\text{out}} - y_k). \tag{12}$$

The physically meaningful root (in $[0, 1]$) is selected; the original input is then recovered as $z = x_k + \xi \, \Delta x_k$. Because the inverse is analytic, it requires no iterative bisection, making example-based training fast even for large datasets.

**Gradient.** Analytical gradients through the softmax/exp re-parameterisations are complex. The app uses central finite differences instead ($\varepsilon = 10^{-4}$), which requires $2(3K+2)$ forward passes per epoch — acceptable because each pass is $O(N)$.

### 3.3 Single-Layer Perceptron (SLP) Transformation

The SLP family combines a linear part with a sum of $K \le 8$ sigmoid units:

$$f_\theta(z) \;=\; a + b\, z + \sum_{k=1}^{K} w_k \, \sigma\!\left( \frac{z - c_k}{s_k} \right), \tag{13}$$

where $\sigma(t) = (1 + e^{-t})^{-1}$ is the logistic sigmoid and $(w_k, c_k, s_k)$ are the weight, centre, and scale of the $k$-th unit. The Jacobian is

$$J_f(z) \;=\; b + \sum_{k=1}^{K} \frac{w_k}{s_k}\, \sigma\!\left(\frac{z - c_k}{s_k}\right)\left[1 - \sigma\!\left(\frac{z - c_k}{s_k}\right)\right]. \tag{14}$$

Because $\sigma(1-\sigma) \geq 0$, enforcing $w_k > 0$ and $b > 0$ guarantees $J_f(z) > 0$ everywhere, making (13) a *strictly monotone* transformation. The SLP can represent asymmetric, multimodal, and sigmoidal reshapings that the cubic polynomial cannot.

The inverse $z = f_\theta^{-1}(x)$ has no closed form for $K \geq 1$; it is computed numerically by bisection (see Section 5.1).

# 4 Training the Flow

## 4.1 Energy-Based Training

Given the target Boltzmann distribution $p^*(x)$, we want the push-forward $p_x$ to approximate $p^*$. A natural objective is the Kullback–Leibler divergence from the model to the target:

$$D_{\mathrm{KL}}(p_x \| p^*) \;=\; \mathbb{E}_{p_x}\!\left[\log \frac{p_x(x)}{p^*(x)}\right] \;=\; \mathbb{E}_{z\sim p_z}\!\left[\frac{U(f_\theta(z))}{k_B T}\right] - \mathbb{E}_{z\sim p_z}[\log |J_f(z)|] + \text{const}, \tag{15}$$

where the constant $\log Z$ is independent of $\theta$. The *variational free energy* (the trainable part) is therefore

$$\boxed{\mathcal{L}_{\text{energy}}(\theta) \;=\; \underbrace{\mathbb{E}_z\!\left[\frac{U(f_\theta(z))}{k_B T}\right]}_{\text{energy term}} \;-\; \underbrace{\mathbb{E}_z[\log |J_f(z)|]}_{\text{entropy term}}.} \tag{16}$$

The energy term penalises configurations in high-potential regions; the entropy term rewards transformations with large Jacobian (i.e. spreading mass). Together they drive the model towards the Boltzmann target.

## 4.2 Example-Based Training (Maximum Likelihood)

When example data $\{x_i\}_{i=1}^N$ from an unknown distribution $p_{\text{data}}(x)$ are available, the parameters can be trained by maximising the log-likelihood of the data under the model, or equivalently by minimising the negative log-likelihood (NLL):

$$\boxed{\mathcal{L}_{\text{NLL}}(\theta) \;=\; \mathbb{E}_{x\sim p_{\text{data}}}[-\log p_z(z) + \log |J_f(z)|\,,}\tag{17}$$

where $z = f_\theta^{-1}(x)$ denotes the pre-image of each data point. The two terms can be interpreted as

- $-\mathbb{E}[\log p_z(z)]$: the latent-space entropy cost — the mapped pre-images should be likely under $p_z$;
- $+\mathbb{E}[\log |J_f(z)|]$: the volume-change cost — a large Jacobian compresses the target density, so the optimiser balances spreading against fitting.

Training example data can be generated from the target Boltzmann distribution via the CDF-inversion trick described in Section 2.3.

## 4.3 Analytical Gradients

Both losses are minimised by stochastic gradient descent with analytical gradients, avoiding the $O(d)$ extra function evaluations of finite differences.

**Energy-based gradient.** For a parameter $\theta_i$,

$$\frac{\partial \mathcal{L}_{\text{energy}}}{\partial \theta_i} \;=\; \mathbb{E}_z\left[\frac{dU}{dx}\,\frac{\partial f_\theta}{\partial \theta_i}\,\frac{1}{k_B T} \;-\; \frac{1}{J_f(z)}\,\frac{\partial J_f}{\partial \theta_i}\right]. \tag{18}$$

**NLL gradient.** Using the implicit-function theorem, $\partial z/\partial \theta_i = -(\partial f_\theta/\partial \theta_i)/J_f(z)$, one obtains

$$\frac{\partial \mathcal{L}_{\text{NLL}}}{\partial \theta_i} \;=\; \mathbb{E}_x\left[\left(s(z) - \frac{\partial_z J_f}{J_f}\right)\frac{\partial f_\theta/\partial \theta_i}{J_f} \;+\; \frac{1}{J_f}\frac{\partial J_f}{\partial \theta_i}\right], \tag{19}$$

where $s(z) = \partial_z \log p_z(z)$ is the *score function* of the latent density. Analytical expressions for $s(z)$ are available for all four latent families.

### 4.4 Parameter Optimisers

Four first-order optimisers are available. In all cases $g$ denotes the current (mini-batch) gradient, $\eta$ the learning rate, and $t$ the epoch counter.

**Stochastic Gradient Descent (SGD).** The simplest update rule: move the parameters a fixed step $\eta$ in the negative gradient direction,

$$\theta \;\leftarrow\; \theta - \eta\, g. \tag{20}$$

Each update uses only the current gradient with no memory of past iterates. Convergence can be slow and oscillatory near valleys where the loss surface curves differently in different directions.

**SGD with momentum.** Accumulates a velocity vector $m$ as an exponential moving average of past gradients,

$$m \;\leftarrow\; \beta_1\, m + g, \qquad \theta \;\leftarrow\; \theta - \eta\, m. \tag{21}$$

The momentum term ($\beta_1 = 0.9$) damps oscillations and accelerates convergence along directions of consistent gradient sign, analogously to a ball rolling with inertia.

**RMSprop.** Maintains a per-parameter running average of squared gradients,

$$v \;\leftarrow\; \beta_2\, v + (1-\beta_2)\, g^2, \qquad \theta \;\leftarrow\; \theta - \frac{\eta}{\sqrt{v+\varepsilon}}\, g. \tag{22}$$

Dividing by $\sqrt{v}$ normalises each parameter's effective step size by the recent gradient magnitude, so parameters with large, noisy gradients take smaller steps while those with small gradients take larger ones. This adaptive scaling is particularly helpful when parameters operate on very different scales. The app uses $\beta_2 = 0.9$, $\varepsilon = 10^{-8}$.

**Adam (default).** Combines momentum and per-parameter adaptive scaling with bias-correction for the first few steps:

$$m \;\leftarrow\; \beta_1\, m + (1-\beta_1)\, g, \qquad \hat{m} = \frac{m}{1-\beta_1^t}, \tag{23}$$

$$v \;\leftarrow\; \beta_2\, v + (1-\beta_2)\, g^2, \qquad \hat{v} = \frac{v}{1-\beta_2^t}, \tag{24}$$

$$\theta \;\leftarrow\; \theta - \frac{\eta\, \hat{m}}{\sqrt{\hat{v}} + \varepsilon}. \tag{25}$$

The bias-corrected estimates $\hat{m}$ and $\hat{v}$ prevent the artificially small steps that would otherwise occur in the first few epochs when $m$ and $v$ are initialised at zero. Adam inherits both the directional smoothing of momentum and the per-parameter scale adaptation of RMSprop, making it robust to a wide range of loss landscape shapes and the recommended default. The app uses $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

# 5 Numerical Methods

## 5.1 Bisection Inversion

The NLL gradient requires evaluating $z = f_\theta^{-1}(x)$ for every data point at every epoch. For the SLP family this is done by bisection on the bracketing interval $[-20, 20]$: since $f_\theta$ is strictly monotone, the midpoint rule halves the interval at each step. Starting from a bracket of width 40, after $n$ steps the residual is at most $40/2^n$; 30 steps reduce it below $4 \times 10^{-8}$, well within the target tolerance of $10^{-7}$.

To avoid paying the bisection cost twice per epoch, the loss and gradient are computed in a single pass: the inverse $z = f_\theta^{-1}(x)$ is found once, the sigmoid activations and Jacobian are accumulated simultaneously, and both the loss and its gradient are evaluated from those shared intermediate quantities.

## 5.2 Example-Data Generation

Exact samples from the Boltzmann target $p^*$ are generated by quantile-transform sampling:

$$x_i^* \;=\; F_{p^*}^{-1}(F_{p_z}(z_i))\,, \tag{26}$$

where $z_i \sim p_z$. Both CDFs are computed numerically on fine grids and the inversions are performed by linear interpolation. This produces samples that are exact (up to grid resolution) without requiring Markov chain Monte Carlo.

# 6 Application Interface

## 6.1 Layout

The browser window is divided into two regions:

- **Left — control panel.** A fixed-width panel with a header image and three tabs (*Densities*, *Map*, *Training*), followed by an auto-rescale checkbox and Reset / Exit buttons. The tabs are shown as a segmented control with icons; the *Exit* button closes the current browser session (it does not stop the server for other users).
- **Right — figure.** A Matplotlib figure split into two columns: the main four-panel display (left) and three histogram / loss panels (right).

## 6.2 Main Four-Panel Display

The panels share their horizontal axis ($z$) and are arranged vertically:

| Panel | Horizontal axis | Vertical axis |
|---|---|---|
| Top strip | $z$ | $p_z(z)$ (latent density, blue) |
| Main panel | $z$ | $x = f_\theta(z)$ (transformation curve, red/orange) |
| Bottom strip | $z$ | $\lvert J_f(z)\rvert^{-1}$ (inverse Jacobian, red/orange) |
| Right strip | $p_x(x)$ (density, green) | $x$ (shared with main) |

Both axes of the main panel carry equal padding of 12 % of the respective data range beyond the displayed support, so the L-shaped mapping lines (see below) have symmetric margins on both sides.

The main panel also shows the target density $p^*(x)$ (amber, when *Show target* is ticked) and the exact transformation $x^*(z)$ (dashed amber, when *Show exact transformation* is ticked). Regions

where $J_f < 0$ are drawn in orange to flag non-invertibility. Enabling *Show target* extends the $x$-range only if the target is *wider* than the transformed distribution; a narrower target leaves the range unchanged (it never causes a spurious rescale).

**Mapping lines.** When *Show mapping lines* is enabled (Densities tab), $N$ evenly spaced values $z_1, \ldots, z_N$ are chosen across the displayed $z$ range and each is connected to its image $x_i = f_\theta(z_i)$ by an L-shaped pair of grey segments:

- a *vertical leg* descending from the top edge of the main panel down to the curve at $(z_i, x_i)$, and
- a *horizontal leg* running from $(z_i, x_i)$ to the right edge of the main panel.

Lines end exactly at the panel borders; no markers are placed on the density curves or on the transformation curve itself. While mapping lines are shown, the horizontal and vertical zero-reference lines in the main panel are suppressed to reduce visual clutter. The number $N$ can be set between 1 and 100; values above 100 are automatically clamped.

### 6.3 Densities Tab

Controls the latent distribution (type and parameters) and the target Boltzmann distribution (temperature $k_B T$, potential coefficients $u_1$–$u_4$).

The $\mu$ and $\sigma$ sliders behave as follows:

- For **Gaussian, Uniform, Laplace, Cauchy**: $\mu$ is the mean (range $[-2, 2]$, default 0) and $\sigma$ is the scale parameter (range $[0.1, 2]$, default 1).
- For **Bimodal**: $\mu$ is the *peak position* — the two peaks sit at $-\mu$ and $+\mu$ — so only positive values are meaningful. The slider range is therefore restricted to $[0, 2]$ with default $\mu = 1$. $\sigma$ is the width of each individual peak (default 0.5). These defaults are applied automatically when Bimodal is selected from the drop-down.

**Show mapping lines** (checkbox + $N=$ entry) — overlays $N \in [1, 100]$ L-shaped mapping lines in the main panel as described in Section 5.2. Entering a value larger than 100 snaps the field back to 100 on the next redraw.

### 6.4 Map Tab

Selects the transformation family (Polynomial, SLP, or RQS) and exposes its parameters via sliders. *Defaults* resets to the near-identity configuration; *Randomise* draws random parameter values.

For the RQS, the number of bins $K \in \{2, 3, 4\}$ is chosen via radio buttons. The sliders expose the unconstrained logits for the bin widths, bin heights, and knot derivatives; the actual constrained values are computed via softmax and exp internally (see Section 3.2). All logits set to zero produce the identity transformation within the support $[-B, B]$.

### 6.5 Training Tab

- **Sample!** — draws $N$ samples from $p_z$ and displays their histogram and the transformed histogram.
- **Generate data** — produces $N$ exact samples from $p^*$ via quantile-transform sampling, for use with example-based training.
- **Mode** — selects Energy-based or Example-based training.

- **Train!** — runs the optimiser in a background thread for the specified number of epochs. A timer animates the transformation in real time at a fixed 30 ms refresh rate. When training finishes the figure shows the *best-loss* parameters encountered during the run (the lowest point of the loss curve), rather than the last iterate — so the final density is never worse than what was displayed while training, even when the loss oscillates near convergence.
- **Epochs / lr / N batch** — total epochs, learning rate, and mini-batch size (Energy-based) or full dataset size (Example-based).
- **Every $n$ ep** — stride between display updates; setting this to a larger value speeds up training at the cost of less frequent animation.
- **Post-training display.** Sliders have finite ranges and silently clamp any trained parameter that falls outside them. To prevent the displayed density from jumping after training ends, the app retains the full-precision parameter vector internally and uses it for all rendering as long as no slider has been manually adjusted since the last training run. Moving any slider or resetting the parameters switches the display back to the slider values.

### 6.6 Right-Column Panels

- **Latent $z$ histogram** — distribution of the current sample / training batch in $z$-space, overlaid with the theoretical $p_z$. Its horizontal axis spans the same $z$-range as the top/main/inverse-Jacobian panels.
- **Transformed $x$ histogram** — same samples pushed through $f_\theta$, overlaid with the theoretical $p_x$ (and the target $p^*$ if *Show target* is on). Its horizontal axis spans the same $x$-range as the main panel's vertical axis. When *Show importance weights* is also ticked, the panel shows the importance-weight overlay described in Section 6.7.
- **Training loss** — the total loss and its two components (energy/latent and entropy/Jacobian terms), drawn in three clearly distinct colours. A fixed subsampling stride (set once at the start of each run) ensures previously plotted points never shift position as new epochs are added.

The density curves overlaid on both histograms (and on the side panels) are drawn without clipping, so their line thickness stays constant even where a curve meets the axis at density zero.

### 6.7 Importance-Weight Overlay

When both *Show target* and *Show importance weights* (Training tab) are ticked and samples are present, the transformed-$x$ histogram panel gains a secondary $y$-axis (right, brown) that shows the *unnormalised importance weights*

$$w(x) \;=\; \frac{p^*(x)}{p_x(x)} \;=\; \frac{p^*(x)\,|J_f(z)|}{p_z(z)}, \qquad z = f_\theta^{-1}(x). \tag{27}$$

A dashed brown curve traces $w(x)$ over the same $x$-range as the density plot; a dotted reference line marks $w = 1$ (perfect agreement between model and target). Where $w > 1$ the model underestimates the target; where $w < 1$ it overestimates it.

The **effective sample size**

$$N_{\text{eff}} \;=\; \frac{\left(\sum_{i=1}^{N} w_i\right)^2}{\sum_{i=1}^{N} w_i^2} \;\in\; (0, N], \tag{28}$$

where $w_i = p^*(x_i)/p_x(x_i)$ at the actual sample points, is shown alongside the *mean importance weight* $\bar{w} = N^{-1}\sum_i w_i$ in the panel annotation.

**Why $N_{\text{eff}}$ alone is insufficient.** $N_{\text{eff}}$ measures *weight uniformity*: it equals $N$ when all $w_i$ are identical, regardless of their magnitude. In particular, *mode collapse* — where $p_x$ concentrates on one mode of a multimodal $p^*$ — produces weights that are all small but equal, so $N_{\text{eff}} \approx N$ even though the model is far from the target.

**Mean weight $\bar{w}$ as an absolute quality indicator.** By the change-of-variables identity $\mathbb{E}_{p_x}[w(x)] = \int p^*(x)\,dx = 1$, the sample mean $\bar{w}$ should be close to 1 for a well-fitted model. With mode collapse, samples land where $p_x \gg p^*$, giving $w_i = p^*/p_x \ll 1$ uniformly, so $\bar{w} \ll 1$.

The two metrics are complementary:

- $\bar{w} \approx 1$: model covers the target correctly (absolute).
- $N_{\text{eff}}/N \approx 100\,\%$: no single sample dominates (relative). A low value flags over-dispersion.

During training, both should approach their ideal values ($\bar{w} \to 1$, $N_{\text{eff}}/N \to 100\,\%$) as the flow converges.

### 6.8 Auto-Rescale Checkbox

When ticked (default), the $z$- and $x$-axis ranges of all panels are recomputed on every redraw to fit the current data. When unticked, those ranges are frozen at their current values, allowing close inspection of a particular region during training or parameter sweeps. The density axes (heights of $p_z$, $|J_f|^{-1}$, $p_x$) continue to rescale freely in both modes.

The axis ranges covered by auto-rescale are: the shared $z$-axis of the top/main/bottom strips; the shared $x$-axis of the main and right strips; the $z$-axis of the latent histogram; and the $x$-axis of the transformed histogram. The transformed-histogram $x$-axis is computed explicitly from the union of all plotted data ranges (samples, target curve, example data) with 4 % padding, ensuring it re-fits correctly when auto-rescale is re-enabled after having been frozen.

## References


[1] D. J. Rezende and S. Mohamed, Variational inference with normalizing flows, *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, PMLR 37:1530–1538, 2015.

[2] G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed, and B. Lakshminarayanan, Normalizing flows for probabilistic modeling and inference, *Journal of Machine Learning Research* **22**(57):1–64, 2021.

[3] F. Noé, S. Olsson, J. Köhler, and H. Wu, Boltzmann generators: Sampling equilibrium states of many-body systems with deep learning, *Science* **365**, eaaw1147, 2019.

[4] A. Coretti, S. Falkner, J. Weinreich, C. Dellago, and O. A. von Lilienfeld, Boltzmann Generators and the new frontier of computational sampling in many-body systems, *KIM Review* **2**, Article 03, 2024.

[5] P. Wirnsberger, A. J. Ballard, G. Papamakarios, S. Abercrombie, S. Racanière, A. Pritzel, D. Jimenez Rezende, and C. Blundell, Targeted free energy estimation via learned mappings, *The Journal of Chemical Physics* **153**, 144112, 2020.

[6] S. Falkner, A. Coretti, S. Romano, P. L. Geissler, and C. Dellago, Conditioning Boltzmann Generators for rare event sampling, *Machine Learning: Science and Technology* **4**, 035050, 2023.

[7] M. Schebek, M. Invernizzi, F. Noé, and J. Rogal, Efficient mapping of phase diagrams with conditional Boltzmann Generators, *Machine Learning: Science and Technology* **5**, 045045, 2024.

[8] M. Schebek, F. Noé, and J. Rogal, Scalable Boltzmann Generators for equilibrium sampling of large-scale materials, *arXiv:2509.25486*, 2025.

[9] A. Coretti, S. Falkner, P. L. Geissler, and C. Dellago, Learning mappings between equilibrium states of liquid systems using normalizing flows, *The Journal of Chemical Physics* **162**, 184102, 2025.

[10] C. Durkan, A. Bekasov, I. Murray, and G. Papamakarios, Neural spline flows, *Advances in Neural Information Processing Systems*, 32, 2019.